
Argentum

MANUAL DE PROYECTO

TALLER DE PROGRAMACIÓN

Integrantes

Thiago Reller	Pad. #111067
Federico Tomás Parsons	Pad. #111250
Agustin Atilio Ferronato	Pad. #111991
Sara Lucia Galeano	Pad. #112120

Índice

1. Introducción	2
2. Evolución del proyecto	3
3. Herramientas y tecnologías	4
3.1. Entorno de desarrollo	4
3.2. Herramientas de calidad de código	4
3.3. Herramientas de diseño gráfico	4
3.4. Librerías externas	4
3.5. Documentación de referencia	4
4. Puntos problemáticos y lecciones aprendidas	5
4.1. Principales dificultades	5
4.2. ¿Qué cambiaríamos si volviéramos a hacerlo?	5
5. Estado del Proyecto: Entrega final	6
5.1. Funcionalidades Completadas	6
6. Conclusión	8

1. Introducción

El presente Manual de Proyecto tiene como objetivo documentar el proceso de desarrollo de Argentum a lo largo de la cursada. En él se describe cómo se organizó el equipo, qué herramientas y tecnologías se utilizaron, cuáles fueron las principales dificultades durante el desarrollo, y qué aprendizajes dejó la experiencia. Además, se detalla el estado actual del proyecto: qué funcionalidades están completas, cuáles están parcialmente implementadas y cuáles quedan pendientes para la próxima entrega.

Cabe aclarar que este manual no cubre aspectos de jugabilidad o controles del juego. Por ejemplo, la interacción con los NPC, uso de comandos por chat entre otros. Para conocer en detalle cómo se juega, los controles, los comandos disponibles, el funcionamiento de la interfaz o funcionalidades que variaron del enunciado, consultar el Manual de Usuario.

2. Evolución del proyecto

Durante la primera semana nos reunimos para discutir la arquitectura general del juego, optando por un esquema cliente-servidor con comunicación a través de un protocolo binario. A partir de allí nos dividimos el trabajo según las preferencias y tiempos de cada integrante.

Planificación inicial vs. real

Nuestra intención era avanzar de forma lineal semana a semana, pero nos encontramos con que la integración de features desarrollados en paralelo requirió refactors más profundos de los anticipados. En particular, la **Semana 2** fue más lenta de lo planeado: la integración del command pattern, la persistencia y el planteo del editor y el mapa requirió varios refactors y coordinación entre los integrantes. Esto nos preocupó porque sentíamos que no estábamos al ritmo esperado. Sin embargo, al redistribuir tareas y compactar esfuerzos en la Semana 3 y 4 logramos recuperar el terreno y completar la mayoría de los objetivos que nos habíamos propuesto para la primera entrega.

Trabajo por semana

- **Semana 1 (12-19 de Mayo):** Planteo de protocolo y parsers, arquitectura cliente-servidor, lobby Qt, SDL con personaje y movimiento.
- **Semana 2 (19-26 de Mayo):** Command pattern, persistencia, planteo del editor, planteo del mapa, colisiones entre jugadores, refactors.
- **Semana 3 (26 de Mayo - 2 de Junio):** Refactor de razas, renderizado por capas, spawn de NPCs, chat, stats, editor con zonas, planteo de inventario e ítems.
- **Semana 4 (2-9 de Junio):** Editor completado, inventario funcional, vestimenta, ataque y defensa, zonas seguras, comandos de chat, música, NPCs interactivos de ciudad.
- **Semana 5-6 (10-22 de Junio):** Archivos de configuración, refactor general de Game.cpp, optimizaciones de renderizado, logica de oro, comando meditar, correcciones, y nuevas animaciones de ataque.

Trabajo por integrante

Esto es una aproximación general a grandes rasgos, pero da una idea en que tuvo foco cada integrante. Esto no quiere decir que cada uno no haya colaborado o aportado ideas en otras áreas.

- **Thiago:** Protocolo, parsers, chat, HUDs, clanes.
- **Federico:** SDL, refactors, razas, ataque, defensa, fair play.
- **Agustin:** Arquitectura cliente-servidor, editor, mapa, ciudades/pueblos, NPCs.
- **Sara:** Lobby Qt, fórmulas de stats, persistencia, inventario, ítems, música.

3. Herramientas y tecnologías

3.1. Entorno de desarrollo

- **IDEs:** CLion, QtCreator, VS Code.
- **Lenguaje:** C++20 con estándar POSIX 2008.
- **Compilador:** g++ (Ubuntu 24.04).
- **Build system:** CMake.
- **Control de versiones:** Git + GitHub con pull requests y revisiones cruzadas.

3.2. Herramientas de calidad de código

- **clang-format:** Formateo automático del código.
- **cpplint:** Estilo Google.
- **cppcheck:** Análisis estático.
- **pre-commit:** Ejecución automática de los chequeos anteriores antes de cada commit.
- **Valgrind:** Detección de fugas de memoria.

3.3. Herramientas de diseño gráfico

- **GIMP:** Edición y modificación de assets gráficos de la interfaz de usuario (HUD, paneles, texturas).
- **Editor de texto:** Ajuste manual de archivos de configuración TOML (layouts, sprites).

3.4. Librerías externas

- **SDL2 + SDL2_mixer:** Gráficos, audio, manejo de eventos.
- **Qt:** Interfaz del editor de mapas.
- **tomlplusplus:** Parseo de archivos de configuración TOML.
- **GoogleTest:** Tests unitarios del protocolo y persistencia.

3.5. Documentación de referencia

- Documentación oficial de SDL2 y SDL2_mixer.
- Documentación de Qt.
- Documentación de tomlplusplus.
- Documentación de GoogleTest.
- cplusplus.com para el estándar C++20.
- Enunciado del TP y hands-on de la cursada.
- Repositorio de recursos gráficos de Argentum Online original.

4. Puntos problemáticos y lecciones aprendidas

4.1. Principales dificultades

- **Integración Qt + SDL2:** Coordinar el bucle de eventos de Qt con el renderizado de SDL2 en el editor requirió varios intentos hasta lograr una solución estable.
- **Refactors del protocolo:** El protocolo se rediseñó varias veces a medida que aparecían nuevos comandos y eventos. Si bien el resultado final es sólido, esto consumió más tiempo del esperado.
- **Dependencias entre features:** Features como ataque/defensa dependían del inventario y las fórmulas de stats, lo que obligó a coordinar merges en un orden específico.
- **Merges conflictivos:** Al trabajar en paralelo sobre las mismas zonas del código (ej. `Game.cpp`), varios merges requirieron resolución manual de conflictos.
- **Performance:** Para la primera entrega tuvimos un porcentaje alto de CPU, lo que obligó a revisar bien el código y buscar cuáles podrían ser las causas de ello. Las optimizaciones de renderizado lograron reducir significativamente la carga.

4.2. ¿Qué cambiaríamos si volviéramos a hacerlo?

- **Definir el protocolo más temprano:** Haber acordado un conjunto estable de comandos y eventos desde la Semana 1 habría evitado varios refactors grandes.
- **Merges más frecuentes:** Integrar cambios a `dev` cada 2-3 días en lugar de una vez por semana habría reducido el tamaño de los conflictos como lo hicimos ya la última semana.
- **Mejor comunicación de dependencias:** Llevar un registro visible de qué features dependen de cuáles para planificar el orden de implementación y tal vez manejar un cronograma desde el inicio.

5. Estado del Proyecto: Entrega final

5.1. Funcionalidades Completadas

Recordar que la jugabilidad, controles de juego y demás están en el Manual de Usuario.

- **Tierras de Argentum:** Mapas, biomas, colisiones (paredes, entidades, jugadores), texturas
- **Puntos de vida/maná:** Fórmulas de HP/Mana max, recuperación, meditación en Formulas.cpp. Comando /meditar implementado.
- **Oro:** Fórmula de oro max, exceso, drop de NPC, pérdida al morir
- **Inventario:** 20 slots, equipar/desequipar items, persistencia
- **Puntos de experiencia:** Fórmulas de XP por ataque y muerte, límites por nivel
- **Ataque:** Daño = Fuerza * rand(dañoArmaMin, max), crítico (doble daño no esquivable), ataque a distancia,
- **Defensa:** Posibilidad de esquivar el ataque, daño que podrá ser reducido según la defensa que le provea su armadura, escudo y/o casco
- **Razas y Clases:** 4 razas (Humano/Elfo/Enano/Gnomo) y 4 clases (Mago/Clerigo/Paladin/Guerrero) con todos los modificadores
- **Muerte y resurrección:** morir, dropear ítems, /resucitar con sacerdote (animación de cuenta regresiva), curarse, teletransporte al sacerdote, no interacción con otros NPCs de ciudad mientras se está muerto.
- **Fair play / Newbie:** protección a nuevos jugadores, diferencia mayor a 10 niveles no permite atacar
- **Ciudades y Pueblos:** Zonas seguras en ciudades, NPC interactivos (por cercanía) reciben y ejecutan comandos correspondientes (sacerdote, comerciante, banquero).
- **Sistema de bancos:** Depósito y retiro de oro e ítems con banqueros en ciudades, persistencia de cuentas.
- **Items:** 19 tipos: espada, hacha, martillo, varas/báculos, arcos, armaduras, cascos, escudos, pociones.
- **NPCs/Criaturas:** 12 tipos con AI que persigue al jugador más cercano, spawn por bioma, límite de población, atacan al jugador y pueden ser atacados por él, además con la muerte de cada criatura existe cierta probabilidad de que esta deje algún objeto.
- **Clanes:** 8 comandos correspondientes y requisitos para ejecutarlos (fundar, unirse, aceptar, rechazar, kick, ban, revisar, salir), persistencia, notificaciones.
- **Interfaz de usuario:** SDL2 con cámara, HUD (HP/mana/oro/XP/nivel), panel de inventario, fullscreen/window
- **Mini-chat:** Overlay de chat con historial, input de texto, mensajes de daño/evasión, mensajes privados entre jugadores
- **Vestimenta:** El personaje cambia visualmente según equipo (arma, armadura, casco, escudo)

- **Persistencia:** Archivos binarios (index + data), guardado periódico cada 300 ticks y al salir
- **Configuración:** TOML Mapas, sprites, layouts, biomas, ítems, NPCs, stats de jugador, datos de tiendas en TOML con tomlplusplus. Patrón Singleton para ItemData.
- **Música y sonido (SFX):** Reproducción de música de fondo (3 tracks) y efectos de sonido (ataque, muerte, curación, etc.) con Audio.cpp + SDL2_mixer
- **Animaciones:** Sprites animados por frames direccionales (caminar/idle), efectos de ataque.
- **Optimizaciones de renderizado:** Renderizado de texturas completas con una sola llamada a `renderer->Copy()`, fondo verde uniforme en lugar de textura de pasto.
- **Protocolo binario:** 25 comandos, 28 eventos, todo serializado/deserializado con parsers registrados por opcode
- **Editor de mapas:** Qt + SDL2 completamente funcional
- **Unit tests:** GoogleTest: protocolo, persistencia, fórmulas, colisiones (42 tests en total)
- **Cheats:** Alejar camara, supervelocidad, vida , mana infinita, morir, obtener items, oro.

6. Conclusión

El desarrollo de Argentum a lo largo de la cursada nos permitió aplicar y consolidar los conceptos fundamentales de la programación con sockets, multithreading, patrones de diseño y trabajo en equipo sobre un código base compartido.

Desde la arquitectura cliente-servidor con protocolo binario hasta la implementación de un editor de mapas completo, pasando por la lógica de combate, la persistencia de datos y las optimizaciones de renderizado, cada etapa representó un desafío que requirió coordinación y toma de decisiones técnicas fundamentadas.

Aunque tuvimos que dedicarle mucho tiempo y esfuerzo el resultado final es un juego multiplayer funcional con la mayoría de las features planteadas en el enunciado, sentando una base sólida sobre la cual agregar nuevas funcionalidades en el futuro, y nos sentimos orgullosos de lo que logramos.